

1 Key Concepts

1.1 Divide and Conquer

The divide-and-conquer paradigm captures a large family of algorithms, for a wide variety of computational problems, that all follow a common template. A divide-and-conquer algorithm for a problem, given an input, *divides* it into smaller input instances of the same problem. It then solves these instances *recursively*, and finally *combines* the solutions in some appropriate way to get a solution for the original input.

Key ingredients. The following ingredients will be present in any divide-and-conquer algorithm. Designing them in the “right” way can often be non-obvious, and requires creative thinking.

1. *Base case.* When the input is small enough, the algorithm just directly outputs an answer. (The base case is often when the input size is zero or one, like sorting an empty or one-element array.)
2. *Divide.* The input is subdivided into some subinputs (which are typically smaller by some constant factor) and the algorithm is called recursively on them. (For example, when the input is an array, it may be split into a “left half” and a “right half.” When the input is a tree, it may be divided into its left and right subtrees.)
3. *Conquer.* The outputs of the recursive calls (on the subinputs) are combined, perhaps together with some processing of the original input, to compute a final output for the full input.

Proving correctness. Proving the correctness of a divide-and-conquer algorithm usually closely mirrors a proof by induction. The proof is structured in two parts:

1. *Base case.* A (usually trivial) proof that the base-case output is correct.
2. *Recursive/inductive step.* A proof that *if* the outputs of the recursive calls are correct, *then* the final output (for the original input) is also correct. This is structured very much like the inductive step of an induction proof: first *assume* that the recursive calls return correct answers, which is known as the *inductive/recursive hypothesis*. Then use this assumption, along with the work performed by the conquer step (and some structural properties of the problem), to show that the final output is correct (for the original input).

By these two steps, it follows that the algorithm is correct on *any* input, of any size. (Formally, this conclusion is reached by *strong induction*; see the discussion in [Handout 2](#) for details.)

Runtime analysis. To analyze the running time $T(n)$ of a divide-and-conquer algorithm, we first derive a *recurrence relation* for it. This is typically an equation that expresses $T(n)$ in terms of values of T for smaller input sizes (representing the running time of the recursive calls), plus some additional term (representing the running time of the conquer step). Often, such a recurrence relation can then be “solved” by invoking the Master Theorem ([Theorem 1](#)), which gives an asymptotic “closed form” for $T(n)$ that is not self-referential.

Many divide-and-conquer algorithms have runtime recurrence relations of the form

$$T(n) = k \cdot T(n/b) + O(n^d),$$

where k is the number of recursive calls the algorithm makes on subinputs, n/b is the size of each subinput, and $O(n^d)$ is the runtime of the conquer step.

For example, MERGESORT has a runtime recurrence of $T(n) = 2T(n/2) + O(n)$, where n represents the number of elements to be sorted. This is because it makes two recursive calls on $n/2$ elements each, then does an $O(n)$ -time MERGE step to interleave (or “merge”) the two sorted results. By the Master Theorem, this has a closed form of $T(n) = O(n \log n)$.

1.2 Master Theorem

The Master Theorem provides “cookbook solutions” for recurrence relations of a certain form. In particular, it provides asymptotic bounds for recurrences that describe the runtime of many divide-and-conquer algorithms you have seen and will see in the future (e.g., Mergesort, Karatsuba multiplication). However, there are also some divide-and-conquer algorithms for which the Master Theorem is not applicable.

Theorem 1 (The Master Theorem). *Let $T(n) = k \cdot T(n/b) + O(n^d)$ for constants $k \geq 1, b > 1, d \geq 0$. Then*

$$T(n) = \begin{cases} O(n^d) & \text{if } k < b^d \\ O(n^d \log n) & \text{if } k = b^d \\ O(n^{\log_b k}) & \text{if } k > b^d. \end{cases}$$

Moreover, if the additive term of the recurrence is a big- Θ expression (rather than a big- O one), then all the closed-form solutions for $T(n)$ are big- Θ bounds as well.

The Master Theorem also generalizes to cases where the additive term (representing the “combine” runtime) has logarithmic factors.

Theorem 2 (The Master Theorem with Log Factors). *Let $T(n) = k \cdot T(n/b) + \Theta(n^d \log^w n)$ for constants $k \geq 1, b > 1, d \geq 0, w \geq 0$. Then*

$$T(n) = \begin{cases} \Theta(n^d \log^w n) & \text{if } k < b^d \\ \Theta(n^d \log^{w+1} n) & \text{if } k = b^d \\ \Theta(n^{\log_b k}) & \text{if } k > b^d. \end{cases}$$

This is a generalization of the Master Theorem in that if $w = 0$, then we obtain the ordinary Master Theorem ([Theorem 1](#)).

2 Discussion Exercises

1. Majority Element

Given an array $A[1, \dots, n]$ of n elements, the *majority element* of A is the element that appears strictly more than $n/2$ times, if such an element exists. Note that if a majority element exists, then it is unique—so we say “*the* majority element” instead of “*a* majority element”. The algorithm MAJORITYELEMENT below outputs the majority element of A if it exists, otherwise it outputs a special symbol $\perp$. For simplicity we assume that n is a power of two, so that we can always split the array exactly in half (and don’t have to deal with annoying floors and ceilings).

```
1: function MAJORITYELEMENT( $A[1, \dots, n]$ )
2:   if  $n = 1$  then return  $A[1]$ 
3:    $x \leftarrow$  MAJORITYELEMENT( $A[1, \dots, n/2]$ )
4:    $y \leftarrow$  MAJORITYELEMENT( $A[n/2 + 1, \dots, n]$ )
5:   if  $x \neq \perp$  then
6:     iterate over  $A$ , counting the number of occurrences of  $x$ 
7:     if the number of occurrences of  $x$  in  $A$  is  $> n/2$  then return  $x$ 
8:   if  $y \neq \perp$  then
9:     iterate over  $A$ , counting the number of occurrences of  $y$ 
10:    if the number of occurrences of  $y$  in  $A$  is  $> n/2$  then return  $y$ 
11:  return  $\perp$ 
```

- (a) Give a recurrence, and asymptotic closed-form solution, for the running time of MAJORITYELEMENT.

Solution: Define $T(n)$ to be the number of steps the algorithm takes on an array of size n . Lines 3 and 4 each recursively call the algorithm on subarrays of size $n/2$, for a total of $2T(n/2)$. Lines 6 and 9 iterate over the entire array, taking $O(n)$ time. The remaining operations take constant time. Thus, we have the recurrence $T(n) = 2T(n/2) + O(n)$. The Master Theorem applies to this with $k = 2, b = 2, d = 1$, and gives us a solution $T(n) = O(n \log n)$.

- (b) Show the correctness of the MAJORITYELEMENT algorithm by first proving the following statement:

If z is the majority element of an array $A[1, \dots, n]$, then z must be the majority element of at least one of the subarrays $A[1, \dots, n/2]$ and $A[n/2 + 1, \dots, n]$.

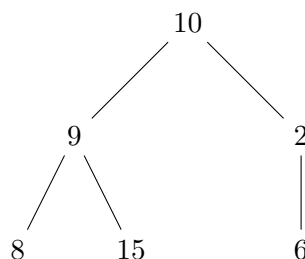
Solution: Suppose that z is the majority element of A , so it appears more than $n/2$ times. This means it must appear *more than $n/4$ times* in *at least one of the two halves* of the array: for if not, it would appear at most $n/4 + n/4 = n/2$ times in the entire array, which violates the assumption. This proves the statement, because each half has $n' = n/2$ elements, and z appears more than $n/4 = n'/2$ times in at least one of the halves.

The statement can be used to prove correctness of the algorithm as follows. The base case is trivial. For the recursive case, the algorithm outputs one of x or y *if and only if* it confirms that the element is indeed the majority element. So, we just need to show that if the majority element of A exists, then it is output by at least one of the recursive calls. Happily, this follows immediately from the given statement—that the majority element of A is also the majority element of at least one of the halves of A —and the recursive hypothesis—that the recursive calls output the majority elements of their respective halves, when they exist.

2. Complete Binary Tree Local Max

A *complete* binary tree is a binary tree in which every level, except possibly the last (leaf) level, is completely filled, and all the nodes in the last level are as far left as possible.

Consider a complete binary tree $T = (V, E, r)$ rooted at r where each vertex is labelled with an integer. A vertex $v \in V$ is called a *local maximum* if the label of v is greater than or equal to the labels of all its adjacent nodes—namely, its parent and children (whichever ones exist).



In this example, the local maxima are 10, 15, and 6.

Suppose you are given such a tree where the labelling is given implicitly, i.e., the only way to determine the label of the vertex v is to visit v and query the label. Give an algorithm that computes a local maximum of T using just $O(\log|V|)$ vertex-label queries. Note: T may contain multiple local maxima; the algorithm just needs to return one of them.

Solution: We provide an algorithm that computes a local maximum of T starting from the root r of T . The main idea is to start from the root r and check if it is a local maximum of T . If so, we are done; if not, we recursively find a local maximum in *just one* of the subtrees of r that is guaranteed to have one, based on the results of the comparisons with r . Specifically, we will show that if the label of r is less than that of one of its children, then the subtree rooted at that child must have a local maximum—so that is the subtree we recurse on.

Input: Complete, rooted, vertex labelled, binary tree $T = (V, E, r)$

Output: Local maximum v^* in T

```

1: function LOCALMAXIMUM( $T = (V, E, r)$ )
2:   if the label of  $r$  is at least the labels of all its children (if any) then
3:     return  $r$ 
4:   else
5:     Let  $r'$  be a child of  $r$  having a label at least as large as  $r$ 's
6:     Let  $T'$  be the (complete, rooted, vertex labeled) binary tree rooted at  $r'$ 
7:     return LOCALMAXIMUM( $T'$ )

```

We show that the algorithm correctly computes a local maximum of T , and that it uses $O(\log|V|)$ vertex label queries.

The algorithm maintains the invariant that the label of the root of its input tree is at least as large as its parent's label (if that parent exists). Additionally, the root is returned if and only if the label of the root is at least as large as all its children's labels. In other

words, the root of the input tree is returned if and only if it is a local maximum. Moreover, the base case is that the tree is just a single (leaf) vertex, in which case the algorithm returns that vertex.

To see that the algorithm uses at most $O(\log|V|)$ vertex-label queries, observe that each invocation of the algorithm uses at most $3 = O(1)$ vertex-label queries (not including what the recursive calls do). Because T is a complete binary tree, there are at most $\lceil \log_2(|V| + 1) \rceil \leq O(\log|V|)$ levels, which is the recursion depth. So, the number of vertex-label queries is indeed $O(\log|V|)$.

3. Divide and Conquer for Integer Multiplication

Karatsuba's algorithm is a fast multiplication algorithm that computes the product of two positive integers x and y using three multiplications of smaller numbers, each with about half as many digits as x and y , plus some additions, subtractions, and digit shifts. In this question, we build on the idea by dividing the numbers into smaller pieces. Namely, we break each argument x, y into *three* equal-sized pieces, of one-third the size. (For simplicity, assume that the size of x and y is a power of three, which can be achieved by "padding" them with some leading zeros; this does not affect the asymptotics.)

Similar to Karatsuba's algorithm, a naive divide-and-conquer algorithm, which we call $\mathcal{A}$, can multiply x and y using **9 recursive multiplications** of numbers that are $n/3$ digits long, along with some linear-time work to combine the results.

Using some clever algebraic manipulations, it is possible to devise a "super-Karatsuba" algorithm that uses only **5 recursive multiplications** of numbers that are about $n/3$ digits long! As before, the results can still be combined in linear time. We call this Algorithm $\mathcal{B}$.

For each algorithm $\mathcal{A}$ and $\mathcal{B}$, derive the recurrence relation for the time complexity $T(n)$, then find an asymptotic closed-form solution for $T(n)$. Which of the algorithms $\mathcal{A}$ and $\mathcal{B}$ is asymptotically faster? How do they compare (asymptotically) to Karatsuba's Algorithm?

Solution: Algorithm $\mathcal{A}$:

- We can see that there are 9 multiplications of numbers that are $n/3$ digits long. The combine-step running time is $O(n)$.
- So, $T(n) = 9T(n/3) + O(n)$.
- By the Master Theorem, since $9 > 3$, $T(n) = O(n^{\log_3 9}) = O(n^2)$.

Algorithm $\mathcal{B}$:

- Similarly, $T(n) = 5T(n/3) + O(n)$.
- By the Master Theorem, since $5 > 3$, $T(n) = O(n^{\log_3 5}) = O(n^{1.465})$.

The algorithm $\mathcal{B}$ runs asymptotically faster. Karatsuba's algorithm runs in time $O(n^{\log_2 3})$. Since $\log_3 5 < \log_2 3$, algorithm $\mathcal{B}$ is asymptotically fastest, Karatsuba's algorithm is the next-fastest, and $\mathcal{A}$ is the slowest (of these three).

Optional content: For those interested, we give the key equations underlying algorithms $\mathcal{A}$ and $\mathcal{B}$ here:

If x, y are n -bit integers, broken up as

$$\begin{aligned}x &= a \cdot 2^{2n/3} + b \cdot 2^{n/3} + c \\y &= d \cdot 2^{2n/3} + e \cdot 2^{n/3} + f\end{aligned}$$

then

$$xy = ad \cdot 2^{4n/3} + (ae + bd)2^n + (af + be + cd)2^{2n/3} + (bf + ce)2^{n/3} + cf = A2^{4n/3} + B2^n + C2^{2n/3} + D2^{n/3} + E,$$

where

$$\begin{aligned}A &= ad \\B &= ae + bd \\C &= af + be + cd \\D &= bf + ce \\E &= cf.\end{aligned}$$

In Algorithm $\mathcal{A}$, computing each value of $ad, ae, af, bd, be, bf, cd, ce, cf$ can be regarded as a sub-problem. We first solve the sub-problems and then compute xy by merging the solutions of sub-problems to get A, B, C, D, E based on the equations above. In Algorithm $\mathcal{B}$, we compute A, B, C, D, E by computing X_0, X_1, X_2, X_3, X_4 below, where as usual the multiplication in each of them is computed recursively:

$$\begin{cases} X_0 = cf \\ X_1 = (a + b + c)(d + e + f) \\ X_2 = (4a + 2b + c)(4d + 2e + f) \\ X_3 = (a - b + c)(d - e + f) \\ X_4 = (4a - 2b + c)(4d - 2e + f) \end{cases} \implies \begin{cases} A = \frac{1}{24}(6X_0 - 4X_1 + X_2 - 4X_3 + X_4) \\ B = \frac{1}{12}(-2X_1 + X_2 + 2X_3 - X_4) \\ C = \frac{1}{24}(-30X_0 + 16X_1 - X_2 + 16X_3 - X_4) \\ D = \frac{1}{12}(8X_1 - X_2 - 8X_3 + X_4) \\ E = X_0 \end{cases}.$$

4. 2D Closest-Pair algorithm

This problem is a review of some key parts of the analysis of the 2D Closest-Pair algorithm from class; it does not contain any new content. The analysis of the algorithm considers a 2δ -by- δ rectangular strip, split by the median line into two δ -by- δ square areas. All points that the algorithm compares in the “left” subset (which lie to the left of, or on, the median line) are at distance at least δ from each other, and similarly, all points in the “right” subset (which must lie to the right of, or on, the median line) are at distance at least δ from each other.

In particular, to show that the algorithm performs better (asymptotically) than the naïve algorithm, we show that the “conquer” portion of this algorithm needs to do only a *constant* number of comparisons per point p . The key fact is that any other point on the “other side,” and within distance δ of p , must be in the bounding rectangle, and only a constant number of points can “fit” within that rectangle.

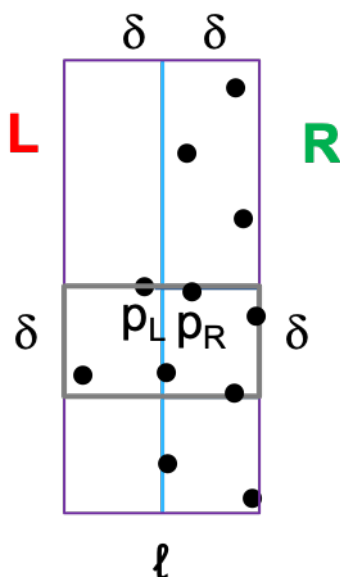


Figure 1: The 2δ -by- δ rectangle (indicated by the gray rectangle) used in the analysis and points within δ of the median line ℓ .

- (a) Prove that the left square area can contain at most 4 points from the left subset, and similarly that the right square area can contain at most 4 points from the right subset.

Hint: partition each square area into four congruent square sub-areas, and show that each sub-area can have at most one point from the relevant subset.

Hint: The pigeonhole principle may be useful (but not necessary) for your proof. It states that if n items are placed in m boxes and $n > m$, then at least one box contains more than one item.

Solution:

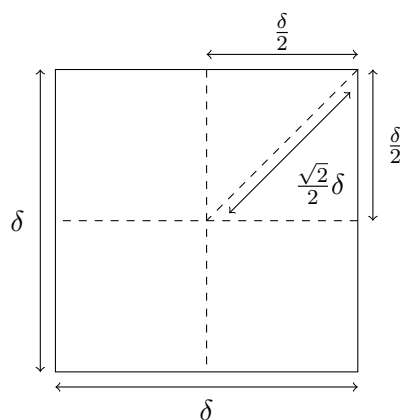


Figure 2: Illustration of the partition

Without loss of generality, consider the $\delta \times \delta$ left square area, whose right edge overlaps with the median line ℓ . We partition the square into four smaller squares, each having side length $\frac{\delta}{2}$.

Suppose, for the purpose of contradiction, that the square contains (at least) 5 points from the left subset. Then by the pigeonhole principle, there exists a smaller square that contains at least two points, possibly on the boundary. However, the farthest apart the two points can be is when they are on the corners, which are at distance $\frac{\sqrt{2}}{2}\delta < \delta$. This is a contradiction, since we know that all points from the left subset are at least δ apart from each other.

- (b) Use the previous part to argue why it suffices for the “conquer” part of the Closest-Pair algorithm to check, for each point in the y -sorted array of points within δ of the median line, only the subsequent 8 points in the array.

(Note: the value 8 here is not optimal, but for the asymptotic conclusion we just need some constant bound.)

Solution: From the “divide” part of the algorithm we already have identified a pair at distance δ , so it suffices for the “conquer” step to find all pairs that are separated by less than δ , if any exist.

Suppose there exist some points p, q where the distance between p and q is less than δ . Then, p and q must have been placed in different subsets (left and right), and therefore must be within distance δ of the median line ℓ . We can assume without loss of generality that p appears before q in the y -sorted array. Since the distance between p and q is less than δ , q must be in the $2\delta \times \delta$ rectangle with p on one of its long horizontal edges (either the bottom or top one, depending on the order in which the points are sorted by y coordinate). This is because the distance between p and q is at least as large as $|p_x - q_x|$ and also as large as $|p_y - q_y|$, by the Pythagorean theorem.

From the previous part, we know that the left and right halves of the rectangle each can have at most 4 points from the left and right subsets (respectively), so the rectangle can contain at most 8 points in total. Therefore, when we consider the 8 points subsequent to p in the sorted list, q must be one of them. Hence, in the conquer step we will consider all pairs of points that are separated by less than δ , which means that we find a closest pair.

3 Additional Problems

1. SlowSort

Consider the sorting algorithm *slowsort*, which can be represented with the following pseudocode. What is the most precise recurrence relation for the running time? Does the Master Theorem apply, and if so, what closed-form solution does it give?

```

1: function SLOWSORT( $A[1, 2, \dots, n]$ ) //  $n$  is length of  $A$ 
2:   SLOWSORT( $A[1, \dots, \lfloor \frac{n}{2} \rfloor]$ ) // sort both halves of the array recursively
3:   SLOWSORT( $A[\lfloor \frac{n}{2} \rfloor + 1, \dots, n]$ )
4:   if  $A[\lfloor \frac{n}{2} \rfloor] > A[n]$  then // largest item in first half is greater than largest in the second
5:     swap  $A[\lfloor \frac{n}{2} \rfloor]$  and  $A[n]$  // put largest item in the unsorted array at the end
6:   SLOWSORT( $A[1, \dots, n - 1]$ ) // recursively sort the entire array minus one element
7:   return

```

- ☐ $T(n) = 2T\left(\frac{n}{2}\right) + T(n - 1) + O(1)$; the Master Theorem gives $T(n) = O(n^2 \log n)$
- ☒ $T(n) = 2T\left(\frac{n}{2}\right) + T(n - 1) + O(1)$; **the Master Theorem does not apply**
- ☐ $T(n) = 3T\left(\frac{n}{2}\right) + O(1)$; the Master Theorem is unusable
- ☐ $T(n) = 3T\left(\frac{n}{2}\right) + O(1)$; the Master Theorem gives $T(n) = O(n^{\log_2 3})$
- ☐ This cannot be written as a relation; the Master Theorem is unusable

Solution: Lines 2 and 3 each take $T\left(\frac{n}{2}\right)$ steps. Line 6 takes $T(n - 1)$ steps. The remaining lines do constant work. The resulting recurrence $T(n) = 2T\left(\frac{n}{2}\right) + T(n - 1) + O(1)$ is not in the form required by the Master Theorem because of the $T(n - 1)$ term.

2. Pikachu: I trade you!

You are a savvy investor who is interested in an especially hot asset that is now on the market: a rare Pikachu Pokémon trading card. You want to purchase and sell this card *just once*, and time these events to maximize your rate of return, i.e., the percentage by which the card's value increases between purchase and sale.

Your top trading-desk researchers have prepared their best estimates of how much the card's value will change on each of the next $n \geq 2$ days. These estimates are given as nonnegative real *multiplicative* factors, i.e., a 10% gain is represented by 1.1, and a 20% loss is represented by 0.8. Given the array $A[1, \dots, n]$ of these estimates, you want to find indices $1 \leq i \leq j \leq n$ that *maximize*

$$A[i] \cdot A[i + 1] \cdots A[j],$$

i.e., the aggregate growth in value if the card is purchased at the *start* of day i and sold at the *end* of day j . Assume that at least one day will have a growth factor greater than 1 (otherwise there would be no reason to buy and sell the card, and you're too savvy for that).

For instance, if $A[1, \dots, 6] = [0.8, 1.1, 0.95, 1.5, 0.8, 1.25]$, then the maximum aggregate growth is 1.5675, which is achieved by $i = 2, j = 6$, and also by $i = 2, j = 4$. So, an optimal choice of i and j is not necessarily unique, but the maximum aggregate growth is unique.

In this question you will design a divide-and-conquer algorithm that solves this problem. You may assume that two real numbers can be multiplied in constant time, and for simplicity, you may assume that n is a power of two (if you wish).

- (a) Suppose that for some day k , where $1 \leq k < n$, you want to optimize the aggregate growth under the constraint that you own the card on day k and sell it strictly after day k —i.e., you must purchase it at the start of some day $i \leq k$ and must sell it at the end of some day $j > k$.

Give an algorithm that solves this problem in $O(n)$ time. On input $A[1, \dots, n]$ and k , it should output the maximum aggregate growth (which might be less than 1 under these constraints), and some corresponding purchase and sale days i, j that achieve it.

Solution: The key idea is to find an optimal subarray $A[i, \dots, k]$ that ends at index k , and an optimal subarray $A[k + 1, \dots, j]$ that begins with index $k + 1$. Then, we return the combined subarray $A[i, \dots, j]$, which we show below is an optimal subarray under the constraint that $i \leq k < j$. This algorithm is made precise in the following pseudocode.

```

1: function MAXGROWTHINCLUDING( $A[1, \dots, n], k$ )
2:    $a \leftarrow A[k], a_{\text{temp}} \leftarrow a, i \leftarrow k$ 
3:   for  $z = k - 1, \dots, 1$  do
4:      $a_{\text{temp}} \leftarrow A[z] \cdot a_{\text{temp}}$ 
5:     if  $a_{\text{temp}} > a$  then  $a \leftarrow a_{\text{temp}}, i \leftarrow z$ 
6:    $b \leftarrow A[k + 1], b_{\text{temp}} \leftarrow b, j \leftarrow k + 1$ 
7:   for  $z = k + 2, \dots, n$  do
8:      $b_{\text{temp}} \leftarrow b_{\text{temp}} \cdot A[z]$ 
9:     if  $b_{\text{temp}} > b$  then  $b \leftarrow b_{\text{temp}}, j \leftarrow z$ 
10:  return ( $g = ab, i, j$ )

```

This algorithm takes $O(n)$ time because there are $O(n)$ iterations, each of which takes only $O(1)$ time.

Correctness: By inspecting the code, it can be seen that the aggregate growth a of $A[i, \dots, k]$ is maximized among all subarrays ending with index k , and the aggregate growth b of $A[k + 1, \dots, j]$ is maximized among all subarrays starting with index $k + 1$. We claim that the aggregate growth $g = ab$ of $A[i, \dots, j]$ is maximized among all subarrays for which $i \leq k < j$. For if not, there is some other subarray $A[i', \dots, j']$ with $i' \leq k < j'$ that has larger aggregate growth $g' > g$. Splitting this subarray, let a' and b' be the aggregate growths of $A[i', \dots, k]$ and $A[k + 1, \dots, j']$, respectively. Then $g' = a'b' > g = ab$, so either $a' > a$ or $b' > b$ (or both). But this contradicts the optimality of either $A[i, \dots, k]$ or $A[k + 1, \dots, j]$ (or both). This completes the proof.

- (b) Now, give a divide-and-conquer algorithm that solves the main problem in $O(n \log n)$ time. Given the array $A[1, \dots, n]$, it should output the optimal aggregate growth and some corresponding purchase and sale days i, j that achieve it.

Solution: The key idea is as follows: we split the input array into two halves. Then, we recursively find an optimal solution within the first half, and similarly for the second half. Then, we use the algorithm from the previous part to find an optimal solution whose starting point is in the first half and whose ending point is in the second half, by setting the argument k to be the midpoint of the array. Finally, we return the best of the three solutions (or some $i > j$ if none of these options are profitable). Precise pseudocode is as follows. (Although this was not required, this code returns both the optimal aggregate growth *and* indices i, j that achieve it, since these are what an investor actually wants to know.)

```

1: function MAXGROWTH( $A[1, \dots, n]$ )
2:   if  $n = 1$  then
3:     return ( $A[1], i = 1, j = 1$ ), or ( $1, i = 1, j = 0$ ) if  $A[1] < 1$ 
4:    $(g_l, i_l, j_l) \leftarrow \text{MAXGROWTH}(A[1, \dots, n/2])$ 
5:    $(g_r, i_r, j_r) \leftarrow \text{MAXGROWTH}(A[n/2 + 1, \dots, n]) + (0, n/2, n/2)$  // shift indices
   to back half of array
6:    $(g_c, i_c, j_c) \leftarrow \text{MAXGROWTHINCLUDING}(A[1, \dots, n], n/2)$ 
7:   return one of the above tuples with largest growth, or ( $1, 1, 0$ ) if all growths
   are less than 1

```

Correctness: For the base case $n = 1$, we can either buy the card at the start of day 1 and sell it at the end of day 1, or not. If we buy and sell, the growth is $A[1]$ and $i = j = 1$. Otherwise, the growth is 1 and $i > j$ to indicate that no trading is done. The algorithm outputs an optimal choice between these two options.

For the recursive case, suppose that i^* and j^* are an optimal choice for the entire array, i.e., the product $A[i^*] \cdot A[i^* + 1] \cdots A[j^*]$ is maximized among all possible choices. There are only four possible cases: (1) $i^*, j^* \leq n/2$, (2) $i^*, j^* > n/2$, (3) $i^* \leq n/2 < j^*$, (4) it is more profitable not to trade at all.

In the first case, the first recursive call will return an optimal solution for the first half, which is therefore optimal for the entire array. (It may not return the exact values i^*, j^* , because there could be other equally profitable choices in the first half, but it will return some optimal pair.) In the second case, the second recursive call will similarly return an optimal solution for the entire array, but with the indices reduced by $n/2$; the “shift” fixes this. In the third case, the call to MAXGROWTHINCLUDING will similarly return an optimal solution. In the fourth case, all of the subroutine calls will return growths less than 1. In every case, our algorithm’s answer is optimal for the entire array.

Running Time: Let $T(n)$ be the algorithm’s running time on an array of n elements. Lines 4 and 5 each take $T(n/2)$ time, while Line 6 takes $O(n)$ time, as shown in the previous part. The remaining lines take $O(1)$ time. Thus, the running time satisfies the recurrence $T(n) = 2T(n/2) + O(n)$, which solves to $T(n) = O(n \log n)$ by the Master Theorem.